

CSC 250: Foundations of Computer Science I

Spring 2026

Chapter 1: Elementary Notations and Notations

Sakhawat Hossan

Department of Computer Science
University of North Carolina at Greensboro

Section 1.1: A Proof Primer

- A proof is a logical argument that shows a statement is true.
- Proofs use clear English sentences combined with mathematical symbols.
- Each step must follow logically from previous steps.
- Why this matters: proofs help us reason correctly and avoid errors.

Statements and Logical Connectives

- In this course, we work with **statements** that are either **true or false**.
- Given two statements **A** and **B**, we form new statements using logical operations:
 - **Negation** If A represents some statement, then the negation of A is the statement “not A,” (*not A*)
 - **Conjunction** The conjunction of A and B is the statement “A and B,” which is true when both A and B are true. (*A and B*):
 - **Disjunction** The disjunction of A and B is the statement “A or B,” which is true if either or both of A and B are true. (*A or B*)

Statements and Logical Connectives

- **Conditionals**

- A conditional statement has the form:
- “If A, then B” (also written as **A implies B**, or **$A \rightarrow B$**)
- A is called the **antecedent**
- B is called the **consequent**

- **Contrapositive**

- The contrapositive of “If A then B” is If NOT B, then NOT A (Written as: $\neg B \rightarrow \neg A$)

A	B	if A then B	if not B then not A
T	T	T	T
T	F	F	F
F	T	T	T
F	F	T	T

Proof Technique Definitions

We will study basic proof techniques using examples involving numbers. The following definitions will be used throughout the course:

Integers:

The set of whole numbers

..., -2, -1, 0, 1, 2

Odd Integers:

Integers of the form $2k+1$, where k is an integer

..., -3, -1, 1, 3, ...

Proof Technique Definitions

Even Integers:

Integers of the form $2k$, where k is an integer
 $\dots, -4, -2, 0, 2, 4, \dots$

Divisibility:

For integers m and n , we say m divides n (written $m \mid n$) if $m \neq 0$ and there exists an integer k such that $n = km$

Prime Numbers:

An integer p is prime if $p > 1$ and its only positive divisors are 1 and p .

Proof Technique Definitions

Converse of a Conditional: Given a conditional statement:

- **If A, then B** ($A \rightarrow B$)

the **converse** is:

- **If B, then A** ($B \rightarrow A$)

The converse is formed by switching the hypothesis and the conclusion.

Important Fact

- A statement and its converse are **not necessarily logically equivalent**.
- A true statement can have a **false converse**.

Example:

Original statement: If a number is divisible by 4, then it is even.

Converse: If a number is even, then it is divisible by 4.

Proof Techniques

Exhaustive Checking

Some statements can be proven by exhaustively checking a finite number of cases.

Example 1. There is a prime number between 200 and 220.

Proof: Check exhaustively and find that 211 is prime.

Conditional Proof

Most statements we prove are conditionals. We start by assuming the antecedent is true.

Then we try to find a statement that follows from the assumption and/or known facts. We continue in this manner until we reach the consequent.

Conditional Proof

Example: If x is odd then x^2 is odd.

Proof: Assume x is odd. Then $x = 2k + 1$ for some integer k .
So we have

$$x^2 = (2k + 1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1,$$

which is an odd integer since $2k^2 + 2k$ is an integer.

If And Only If (Iff) Proofs

A statement of the form “ A if and only if B ” means “ A implies B ” and “ B implies A .” ($A \iff B$)

Example:

Claim: An integer x is even **if and only if (iff)** $x^2 - 2x + 1$ is odd.

Proof:

x is even iff $x = 2k$ for some integer k (definition)

iff $x - 1 = 2k - 1$ for some integer k (algebra)

iff $x - 1 = 2(k - 1) + 1$ for some integer $k - 1$ (algebra)

iff $x - 1$ is odd (definition)

iff $(x - 1)^2$ is odd

iff $x^2 - 2x + 1$ is odd (algebra).

Proof By Contradiction

A false statement is called a *contradiction*.

For example, “S and not S” is a contradiction for any statement S.

A **proof by contradiction** is a proof technique in which we assume that the statement to be proved is **false**, and then logically derive a **contradiction**. Since a statement and its negation cannot both be true, the contradiction shows that the original statement must be **true**.

In practice, to prove a statement of the form
P is true,

we assume

$$\neg P,$$

Proof By Contradiction

Example: If x^2 is odd, then x is odd.

Proof (By Way of Contradiction)

Assume, **by way of contradiction**, that x^2 is odd **and** x is even.

Since x is even, there exists an integer k such that

$$x = 2k.$$

Then

$$x^2 = (2k)^2 = 4k^2 = 2(2k)^2$$

which is even.

This contradicts the assumption that x^2 is odd.

Therefore, our assumption that x is even must be false. Hence, x is odd. $\square$

Proof Exercise

1. We Let S be the statement, “If x is even, then $x + 1$ is odd.”
 - a. Write the converse of S .
Answer: If $x + 1$ is odd, then x is even.
 - b. Write the contrapositive of S .
Answer: If $x + 1$ is even, then x is odd.
2. Prove the converse of each statement.
 - a. If x is even, then xy is even for any integer y .
 - b. If y is odd, then $x(x + y)$ is even for any integer x
3. Give an indirect proof (by contradiction) of the following statement about integers: The product $x(x + 1)$ is even for every integer x .

Section 1.2 Sets

A **set** is a collection of things.

If S is a set and x is a member or element of S we write $x \in S$. Otherwise, we write $x \notin S$.

- The empty set with no elements is denoted $\{ \}$ or $\emptyset$.
- A set with one element is called a singleton. e.g., $\{a\}$ is a singleton.
- The set of *integers* is denoted by **Z**, the *natural numbers* $\{0, 1, \dots\}$ by **N**, the rational numbers by **Q**, and the real numbers by **R**.

Equal Sets

Two sets A and B are **equal**, denoted $A = B$ if they have the same elements.

e.g., $\{a, b, c\} = \{c, b, a\}$ (no ordering).

e.g., $\{a, a, b, c\} = \{a, b, c\}$ (no repetitions)

Subsets

The set A is a **subset** of B , denoted $A \subseteq B$, means every element of A is an element of B . $A \subseteq B$ allows equality, but $A \subset B$ means A is strictly smaller than B .

e.g., $\mathbf{N} \subseteq \mathbf{Z} \subseteq \mathbf{Q} \subseteq \mathbf{R}$.

e.g., $S \subseteq S$ for any set S .

e.g., $\emptyset \subseteq S$ for any set S .

Subsets

Example(Comparing sets). Let $A = \{2k + 7 \mid k \in \mathbf{Z}\}$ and $B = \{4k + 3 \mid k \in \mathbf{Z}\}$.

Quiz. Is $A \subseteq B$?

Answer: No. For example, $9 \in A$ but $9 \notin B$.

Quiz. Is $B \subseteq A$?

Answer: Yes. Let $x \in B$. Then $x = 4k + 3$ for some $k \in \mathbf{Z}$. But we can write

$$x = 4k + 3 = 4k - 4 + 7 = 2(2k - 2) + 7.$$

Since $2k - 2 \in \mathbf{Z}$, it follows that $x \in A$. Therefore $B \subseteq A$.

Equality

Prove $A = B$ iff $A \subseteq B$ and $B \subseteq A$.

Example: Let $A = \{2k + 5 \mid k \in \mathbf{Z}\}$ and $B = \{2k + 3 \mid k \in \mathbf{Z}\}$.

Show that $A = B$.

Proof:

First, we show $A \subseteq B$.

Let $x \in A$. Then $x = 2k + 5$ for some $k \in \mathbf{Z}$. So, we have

$$x = 2k + 5 = 2k + 2 + 3 = 2(k + 1) + 3.$$

Since $k + 1 \in \mathbf{Z}$, it follows that $x \in B$. Therefore $A \subseteq B$.

Equality

Now we show the other direction $B \subseteq A$.

Let $x \in B$.

Then $x = 2k + 3$ for some $k \in \mathbf{Z}$.

We rewrite x as follows:

$$x = 2k + 3 = 2k - 2 + 5 = 2(k - 1) + 5.$$

Since $k - 1 \in \mathbf{Z}$, it follows that $x \in A$.

Therefore,

$$B \subseteq A.$$

Since $A \subseteq B$ and $B \subseteq A$ it follows that $A = B$.

Power Set

The **power set** of a set S , denoted $\text{power}(S)$ is the set of all subsets of S .

e.g., $\text{power}(\{a, b\}) = \{\emptyset, \{a\}, \{b\}, \{a, b\}\}$.

Operations on Sets

- Union: $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$.
- Intersection: $A \cap B = \{x \mid x \in A \text{ and } x \in B\}$.
- Difference: $A - B = \{x \mid x \in A \text{ and } x \notin B\}$.

Operations on Sets

- Symmetric Difference: $A \oplus B = \{x \mid x \in A \text{ or } x \in B \text{ but not both}\}$ Note: $A \oplus B = (A - B) \cup (B - A) = (A \cup B) - (A \cap B)$.
- Universal Complement: Given a universe U and $A \subseteq U$, we write
$$A' = U - A.$$

Operations on Sets Examples

• *Example.*

$D_0 = \{1, 2, 3, \dots\}$, $D_5 = \{1, 5\}$, $D_6 = \{1, 2, 3, 6\}$, and $D_9 = \{1, 3, 9\}$.

$$D_5 \cup D_6 = \{1, 2, 3, 5, 6\}$$

$$D_5 \cap D_6 = \{1\}$$

$$D_9 - D_6 = \{9\}$$

$$D_5 \oplus D_6 = \{2, 3, 5, 6\}$$

Let $\mathbf{N}$ be the universe. Then $D_0' = \mathbf{N} - D_0 = \{0\}$, and $\{0\}' = D_0$.

Operations on Sets Examples

- Difference of Sets $A - B = \{x \mid x \in A \text{ and } x \notin B\}$.
- For example, if $A = \{a, b, c\}$ and $B = \{c, d\}$, then $A - B = \{a, b\}$.
We can picture the difference $A - B$ of two general sets A and B by the shaded region of the Venn diagram in Fig 1.
- Symmetric Difference: $A \oplus B = \{x \mid x \in A \text{ or } x \in B \text{ but not both}\}$ For example, if $A = \{a, b, c\}$ and $B = \{c, d\}$, then $A \oplus B = \{a, b, d\}$.
The set $A \oplus B$ is represented by the shaded regions of the Venn diagram in Fig 2.

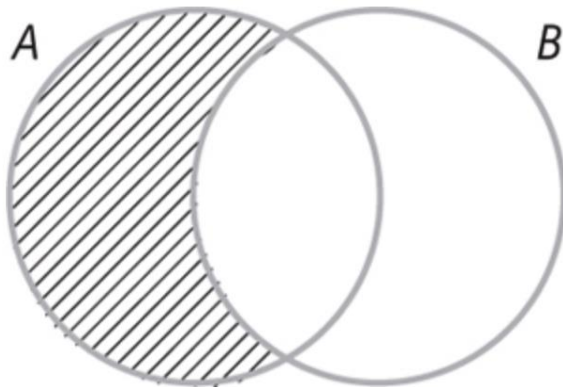


Figure 1: $A - B$

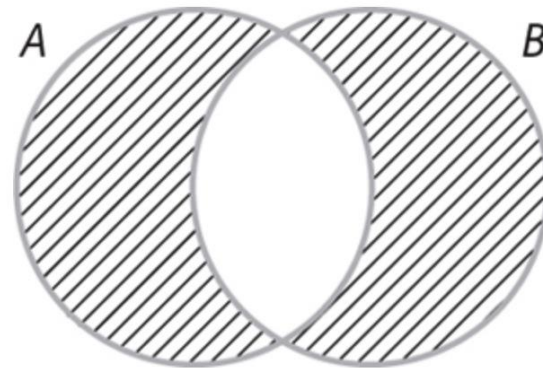


Figure 1: $A \oplus B$

Combining Properties of Union and Intersection

- a. $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$. ($\cap$ distributes over $\cup$.)
- b. $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$. ($\cup$ distributes over $\cap$.)
- c. $A \cap (A \cup B) = A$. (absorption law)
- d. $A \cup (A \cap B) = A$. (absorption law)

A Distributive Proof:

We'll prove the following statement: $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$.

Proof: We'll show that $x \in A \cap (B \cup C)$ if and only if $x \in (A \cap B) \cup (A \cap C)$.

$x \in A \cap (B \cup C)$ iff $x \in A$ and $x \in B \cup C$

iff $x \in A$, and either $x \in B$ or $x \in C$

iff either $(x \in A \text{ and } x \in B)$ or $(x \in A \text{ and } x \in C)$

iff $x \in (A \cap B) \cup (A \cap C)$.

Set Exercises

1. Find the set A that satisfies the following four properties:

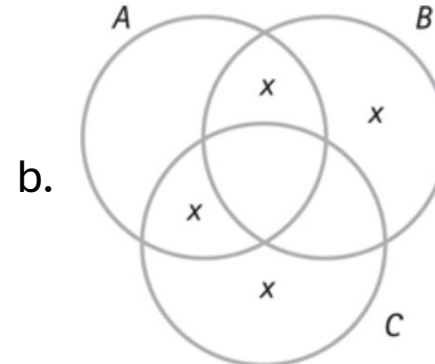
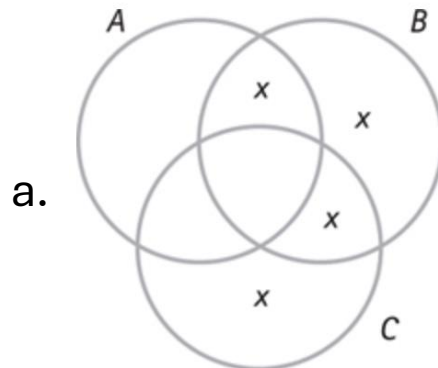
$$A \cap \{2, 3, 5, 6\} = \{2, 5\}$$

$$A \cup \{4, 5, 6, 7\} = \{1, 2, 4, 5, 6, 7\}$$

$$\{2, 4\} \subseteq A$$

$$A \subseteq \{1, 2, 4, 5, 6, 8\}.$$

2. Each Venn diagram in the following figure represents a set whose regions are indicated by the letter x . Find an expression for each of the three sets in terms of set operations. Try to simplify your answers.



Set Exercises

3. Prove that $\text{power}(A \cap B) = \text{power}(A) \cap \text{power}(B)$.
4. Prove each of the following properties of the complement
 - a. $(A')' = A$.
 - b. $\emptyset' = U$ and $U' = \emptyset$.
 - c. $A \cap A' = \emptyset$ and $A \cup A' = U$.
 - d. $(A \cup B)' = A' \cap B'$. (DeMorgan's law)
 - e. $(A \cap B)' = A' \cup B'$. (DeMorgan's law)
 - f. $A \cap (A' \cup B) = A \cap B$. (absorption law)

Section 1.3: Ordered Structures

What does “ordered” mean?

- In ordered structures, position matters

Tuples

- A tuple is an ordered collection of elements
- Order matters
- Repetition is allowed

Examples

- $(6, 7, 6)$ is a 3-tuple
- $()$ is the empty tuple

Equality of tuples

- $(x_1, \dots, x_n) = (y_1, \dots, y_n)$ iff $x_i = y_i$ for all i

Cartesian Product

Definition

- $A \times B = \{(x, y) \mid x \in A \text{ and } y \in B\}$
- Produces ordered pairs
- Order is important: $(x, y) \neq (y, x)$

Extension

- $A \times B \times C = \{(x, y, z) \mid x \in A, y \in B, z \in C\}$

Example:

if $A = \{x, y\}$ and $B = \{0, 1\}$, $C = \{x\}$

Then, $A \times B = \{(x, 0), (x, 1), (y, 0), (y, 1)\}$.

$A \times B \times C = \{(1, a, x), (1, b, x), (2, a, x), (2, b, x)\}$

One-Dimensional Arrays

A one-dimensional array of size n with elements from a set A can be viewed as an n -tuple.

Formally:

$$A^n = \{(x_1, x_2, \dots, x_n) \mid x_i \in A\}$$

- Each element has a fixed position
- Order matters
- Repetition is allowed

If

$$x = (x_1, x_2, \dots, x_n),$$

then in programming languages, the component x_i is written as:
 $x[i]$

Two-Dimensional Arrays (Matrices)

A two-dimensional array, also called a matrix, is a table of elements indexed by:

- rows
- columns

If a matrix has:

- m rows
- n columns

we call it an $m \times n$ matrix.

Each entry is identified by two indices.

Example: A 3×3 matrix

$$X = \begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix}$$

Lists

A **list** is similar to a tuple but is treated differently operationally.

- Lists are ordered
- Repetition is allowed
- Lists are processed from the front

Examples:

- $\langle a, b, a, c \rangle$ is a list
- $\langle \rangle$ is the empty list

Lists are common in functional programming.

Strings

A string is a special kind of list whose elements come from a fixed alphabet.

- Written by juxtaposition (no commas)
- Order matters
- Repetition allowed

Example:

If $A = \{a, b\}$, then strings include:

$a, b, aa, ab, ba, bb, aaa, \dots$

Empty String and Concatenation

- The empty string is denoted by ε
- Concatenation joins strings end to end

Examples:

- $ab \cdot bab = abbab$
- $s\varepsilon = \varepsilon s = s$

If s is a string, then:

- s^n means s concatenated with itself n times
- $s^0 = \varepsilon$

Languages

A language is a set of strings, usually over a fixed alphabet.

Notation:

- If A is an alphabet, then A^* is the set of **all strings** over A

Examples of languages:

- $\emptyset$
- $\{\epsilon\}$
- A
- A^*

Example Language

- $\{ab^n a \mid n \in \mathbb{N}\}$

This language contains:

aa, aba, abba, abbba, ...

The description defines an infinite language using a pattern.

- A^*

Let:

$$A = \{a' b\}$$

Then:

$$A^* = \{\varepsilon' a' b' a' a' a' b' b' a' bbaaaaab \dots\}$$

This set is **infinite**, even though A is finite.

Language Operations

Let L and M be languages.

Product (Concatenation)

$$LM = \{s' t \mid s \in L, t \in M\}$$

Powers

- $L^0 = \{\varepsilon\}$
- $L^n = \{s_1 \dots s_n \mid s_i \in L\}$

Kleene Star (Closure)

The **closure** of a language L is:

$$L^* = L^0 \cup L^1 \cup L^2 \cup \dots$$

Language Operations

Important results:

- $\emptyset^* = \{\varepsilon\}$
- $\{\varepsilon\}^* = \{\varepsilon\}$

1.4 Graphs and Trees

A graph is a mathematical structure used to model relationships between objects.

A graph G consists of:

A set of vertices (nodes), denoted V

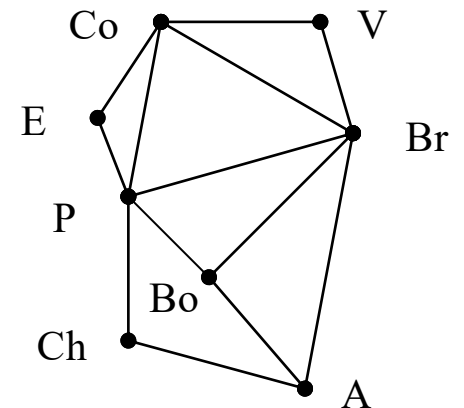
A set of edges, denoted E

Formally:

$$G = (V, E)$$

Vertices represent objects

Edges represent connections or relationships between objects



Vertices and Edges

Vertices (Nodes)

The **individual elements** of a graph Represent entities such as cities, users, webpages, or tasks

Example:

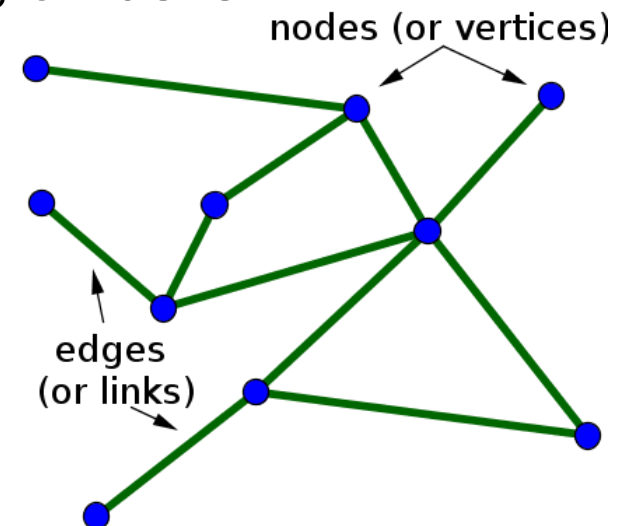
$$V = \{A, B, C, D\}$$

Edges

- Connections between vertices
- May represent roads, friendships, communication links, etc.

An edge connecting vertices u and v is written as:

- $\{u, v\}$ in an undirected graph
- (u, v) in a directed graph



Directed and Undirected Graphs

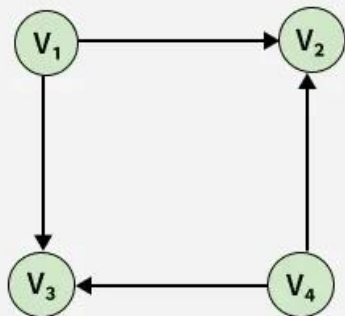
Directed Graph (Digraph)

- Edges have a **direction**
- Written as ordered pairs

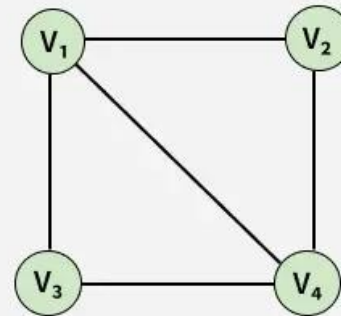
Undirected Graph

- Edges have **no direction**
- Connection works both ways

Directed Graph



Undirected Graph



Paths, Shortest Path, and Longest Path

Path

A path is a sequence of vertices where each consecutive pair is connected by an edge.

Shortest Path

A path with the minimum number of edges (or minimum total weight)

- Common in navigation and routing problems

Longest Path

A path with the maximum number of edges

- Much harder to compute in general

Algorithms for Shortest Paths

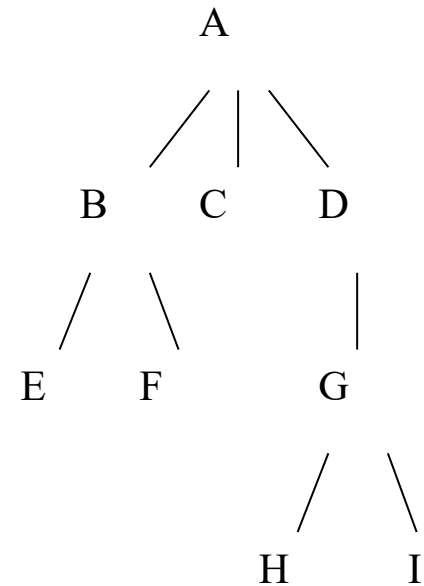
- **Breadth-First Search (BFS)**
 - Used when all edges have equal weight
- **Dijkstra's Algorithm**
 - Used when edges have non-negative weights
- **Bellman–Ford Algorithm**
 - Allows negative edge weights

Trees

A tree is a connected graph (a path between any two points) with no cycles. Most trees are oriented so that they look like upside-down trees, such as the tree pictured.

Example:

For this tree the root is A. The children of A are B, C, D. D is the parent of G. The height or depth of the tree is 3. The leaves are E, F, C, H, I.

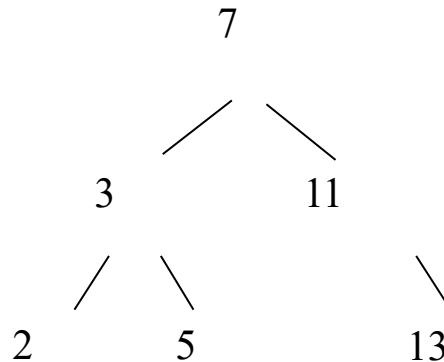


The pictured tree can be represented by the list

$\langle A, \langle B, \langle E \rangle, \langle F \rangle \rangle, \langle C \rangle, \langle D, \langle G, \langle H \rangle, \langle I \rangle \rangle \rangle \rangle$.

Binary Trees

A binary tree is either empty, denoted by $\langle \rangle$, or each node has two subtrees that are binary trees and are called the left and right subtrees of the node. If a binary tree is not empty, we'll represent it as a list of the form $\langle L, x, R \rangle$, where x is the root and L and R are the left and right subtrees, respectively.



Example: $\langle \langle \langle 2 \rangle, 3, \langle 5 \rangle \rangle, 7, \langle, 11, \langle 13 \rangle \rangle \rangle$

Dijkstra's Algorithm

Purpose

- **Dijkstra's Algorithm** is used to find the **shortest path** from a starting vertex to **all other vertices** in a graph.
- Works on **weighted graphs**
- All edge weights must be **non-negative**
- Commonly used in **routing, navigation, and networks**

Algorithm Steps

Given a weighted graph and a start vertex s :

1. Assign a distance of **0** to s , and ∞ (**infinity**) to all other vertices.
2. Mark all vertices as **unvisited**.

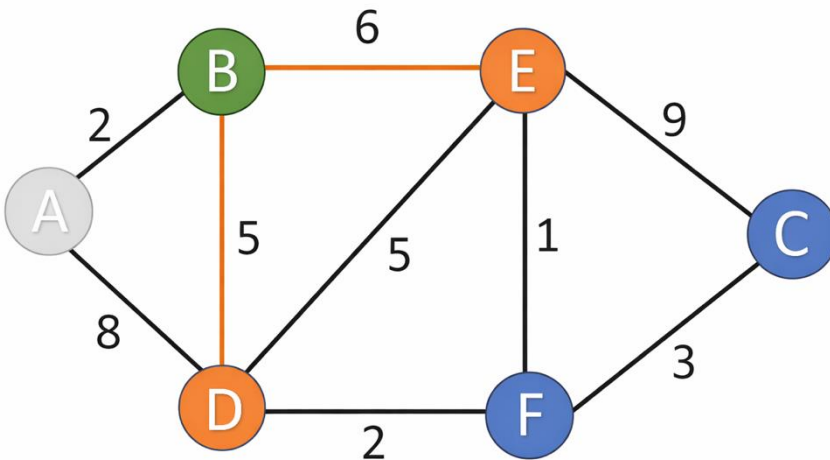
Dijkstra's Algorithm

- 3) Select the unvisited vertex with the **smallest known distance**.
- 4) For each neighbor of this vertex:
 - Compute a new possible distance

$$\text{new distance} = \text{current distance} + \text{edge weight}$$

- If the new distance is smaller, **update** it.
- 5) Mark the current vertex as **visited**.
 - 6) Repeat steps 3–5 until all vertices are visited (or the destination is reached).

Dijkstra's Algorithm Example



Node	Shortest Distance	Previous Node
A	0	—
B	2	A
C	12	F
D	7	B
E	8	B
F	9	D

Initialization:

Set $dist(A) = 0$

Set all others to ∞

Previous node = blank initially

Dijkstra's Algorithm Example

Step 1: Visit A (distance 0)

Relax neighbors of A:

- $B: 0 + 2 = 2 \rightarrow \text{update } \text{dist}(B) = 2, \text{prev}(B)=A$
- $D: 0 + 8 = 8 \rightarrow \text{update } \text{dist}(D) = 8, \text{prev}(D)=A$

Mark **A** visited.

Step 2: Visit B (smallest unvisited distance = 2)

Relax neighbors of B:

- $D: 2 + 5 = 7 \rightarrow \text{better than } 8 \rightarrow \text{update } \text{dist}(D) = 7, \text{prev}(D)=B$
- $E: 2 + 6 = 8 \rightarrow \text{update } \text{dist}(E) = 8, \text{prev}(E)=B$

Mark **B** visited.

Dijkstra's Algorithm Example

Step 3: Visit D (smallest unvisited distance = 7)

Relax neighbors of D:

- $F: 7 + 2 = 9 \rightarrow$ update $dist(F) = 9$, $prev(F)=D$
- $E: 7 + 3 = 10 \rightarrow$ not better than 8 $\rightarrow$ no change

Mark **D** visited.

Step 4: Visit E (smallest unvisited distance = 8)

Relax neighbors of E:

- $F: 8 + 1 = 9 \rightarrow$ equals current 9 $\rightarrow$ keep as is (prev stays D)
- $C: 8 + 9 = 17 \rightarrow$ update $dist(C) = 17$, $prev(C)=E$

Mark **E** visited.

Dijkstra's Algorithm Example

Step 5: Visit F (smallest unvisited distance = 9)

Relax neighbors of F:

- $C: 9 + 3 = 12 \rightarrow$ better than 17 $\rightarrow$ update $dist(C) = 12$,
prev(C)=F

Mark **F** visited.

Step 6: Visit C (distance 12)

- Done.